



INSTITUTO FEDERAL
Rio Grande do Sul

Banco de Dados II

Estruturas indexação

Prof. Edimar Manica

edimar.manica@ibiruba.ifrs.edu.br



Referência

- Referência
 - Título: Sistemas de Banco de Dados
 - Autores: Elmasri & Navathe
 - Edição: 6^a
 - Capítulo: Estruturas de indexação para arquivos
 - Páginas: 424-455

Onde paramos?

- Os registros (tuplas das relações) são armazenados em arquivos
- Os arquivos possuem alguma organização primária (forma como os registros estão no arquivo que está no disco)
 - Desordenada
 - Ordenada
 - Hashed

Roteiro

- Tipos de Índices Ordenados de Único Nível
 - Índices primários
 - Índices de agrupamento
 - Índices secundários
- Índices multiníveis
- Índices multiníveis dinâmicos usando B Trees e B+ Trees

O que veremos?

- Índices

- O que são?

- Logicamente: Estruturas de acesso auxiliares
 - Fisicamente: Arquivos adicionais no disco

- Para que?

- Agilizar a recuperação de registros em resposta a certas condições de pesquisa
 - Oferecer formas eficientes de acessar os registros sem afetar seu posicionamento físico no arquivo de registros no disco

Índices

- Analogia
 - Materiais
 - Um livro
 - Atividade
 - Encontrar as páginas que contenham uma determinada palavra
 - Objetivo
 - Analisar a diferença entre a pesquisa linear e a pesquisa indexada

Índices como caminhos de acesso

- Um índice de nível único é um arquivo auxiliar que torna a busca mais eficiente para um registro em um arquivo de dados
- O índice é normalmente especificado em um campo no arquivo (embora ele possa ser especificado em vários campos)
- Uma forma de um índice é um arquivo de entradas <**valor do campo, ponteiro para o registro**>, que é ordenado pelo *valor do campo*
- O índice é chamado de um caminho de acesso no campo

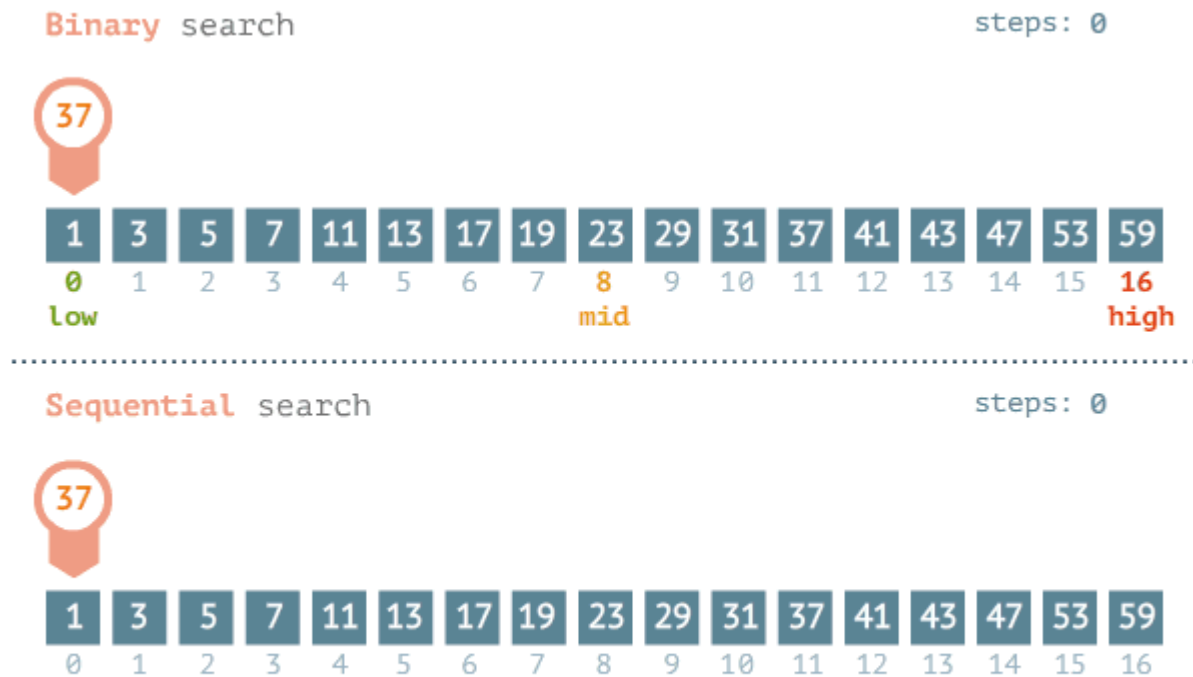


Índices como caminhos de acesso

- O arquivo de índice normalmente ocupa consideravelmente menos blocos de disco que um arquivo de dados porque suas entradas são muito menores
- Os valores no índice são ordenados de modo que possamos realizar uma pesquisa binária



- <https://blog.penjee.com/wp-content/uploads/2015/04/binary-and-linear-search-animations.gif>



www.penjee.com

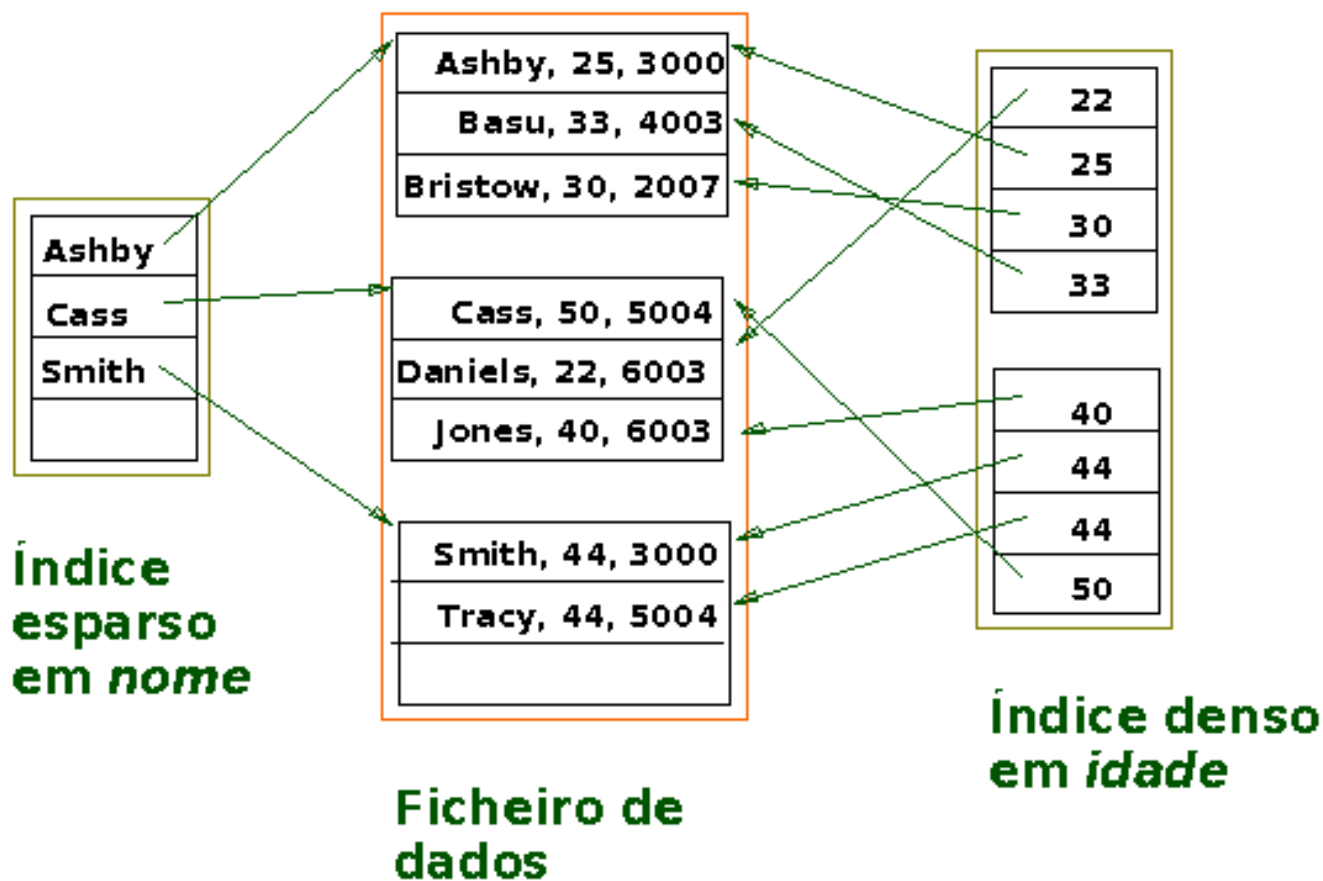


INSTITUTO FEDERAL
Rio Grande do Sul

Índices como caminhos de acesso

- Índices podem ser caracterizados como densos ou esparsos
 - Um **índice denso** tem uma entrada para cada valor de chave de pesquisa (e, portanto, cada registro) no arquivo de dados.
 - Um **índice esparsos** (ou **não denso**) possui entradas para somente alguns dos valores de pesquisa





Exemplo de pesquisa linear (sem índices)

- Exemplo: Dado o seguinte arquivo de dados **ordenado**
 - EMPREGADO (NOME, ID, ENDEREÇO, FUNÇÃO, SALÁRIO, ...)
- Considere que:
 - Tamanho do arquivo $r=30.000$ registros
 - Tamanho do bloco $B=1.024$ bytes
 - Os registros são de tamanho fixo e não espalhados, com tamanho $R=100$ bytes
- Então, nós temos:
 - Fator de bloco $Bfr = B \div R = 1.024 \div 100 = 10$ registros por bloco
 - Número de blocos para o arquivo $b = (r/Bfr) = (30.000/10) = 3.000$ blocos
- Pesquisa linear sem índices (caso médio)
 - $(b/2) = 3.000/2 = 1.500$ acessos de bloco

Exemplo de pesquisa binária (sem índices)

- Exemplo: Dado o seguinte arquivo de dados **ordenado**
 - EMPREGADO (NOME, ID, ENDEREÇO, FUNÇÃO, SALÁRIO, ...)
- Considere que:
 - Tamanho do arquivo $r=30.000$ registros
 - Tamanho do bloco $B=1.024$ bytes
 - Os registros são de tamanho fixo e não espalhados, com tamanho $R=100$ bytes
- Então, nós temos:
 - Fator de bloco $Bfr = B \div R = 1.024 \div 100 = 10$ registros por bloco
 - Número de blocos para o arquivo $b = (r/Bfr) = (30.000/10) = 3.000$ blocos
- Pesquisa binária sem índices
 - $\lceil \log_2 b \rceil = \lceil (\log_2 3.000) \rceil = 12$ acessos de bloco

Exemplo de pesquisa binária com índices

- Considere que:
 - Tamanho do arquivo $r=30.000$ registros
 - Tamanho do bloco $B=1.024$ bytes
- Para um índice sobre o campo ID, assumindo que o tamanho do campo ID é $V=9$ bytes e o tamanho do ponteiro de registro é $P=6$ bytes. Então:
 - Tamanho de cada entrada de índice $R_I=(V+P)=(9+6)=15$ bytes
 - Fator de bloco p/ o índice
 - $Bfr_I = B \text{ div } R_I = 1.024 \text{ div } 15 = 68$ entradas de índice por bloco
 - O número total de entradas de índice r_I é igual ao número de blocos no arquivo de dados = 3.000 (b calculado no slide anterior)
 - Número de blocos de índice $b_I = (r_I / Bfr_I) = (3.000 / 68) = 45$ blocos
 - Busca binária precisa de $\log_2 b_I = \log_2 45 = 6$ acessos de bloco
 - Para procurar um registro usando o índice, precisamos de um acesso de bloco adicional (a partir do índice buscar no arquivo através do ponteiro)
 - **Pesquisa binária com índices**
 - Total de acessos a bloco de disco = $6+1 = 7$

Resumo

- Pesquisa linear sem índices (caso médio)
 - 1.500 acessos de bloco
- Pesquisa binária sem índices
 - 12 acessos de bloco
- Pesquisa binária com índices
 - 7 acessos de bloco



Tipos de índices de nível único

- Índice primário
- Índice de agrupamento
- Secundário

Índice primário

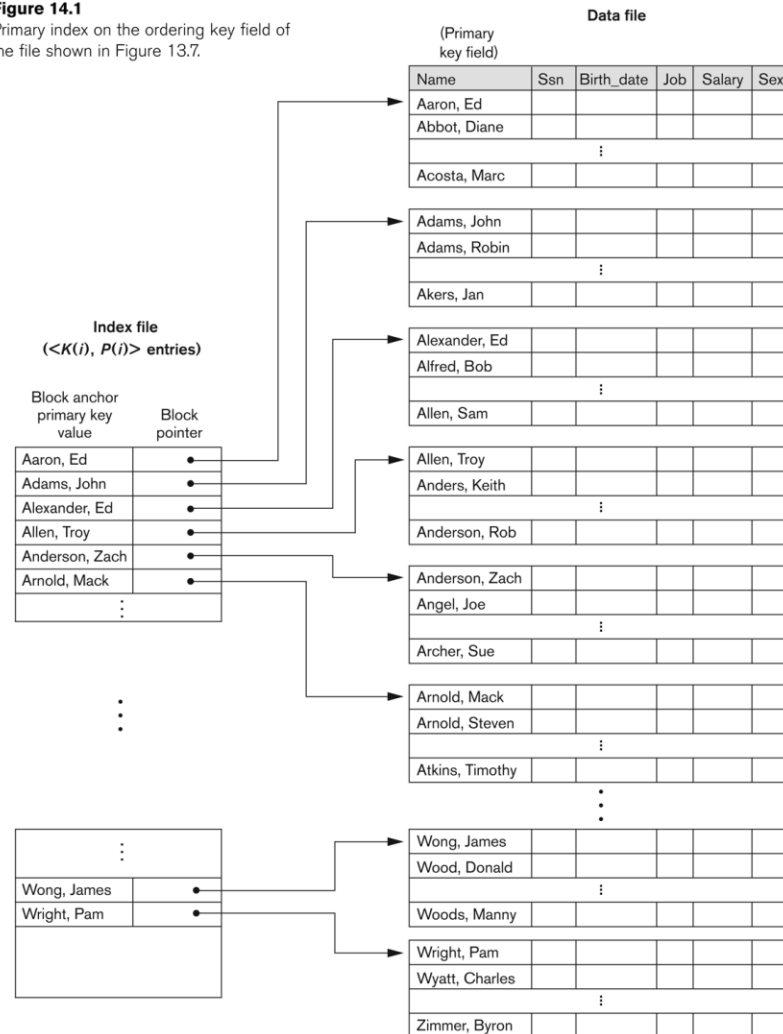
- Definido sobre um arquivo de dados ordenado
- O arquivo de dados está fisicamente ordenado em um **campo chave** (que possui um valor distinto para cada registro)
- Inclui uma entrada de índice para cada bloco no arquivo de dados
- A entrada de índice tem o valor do campo chave do primeiro registro no bloco, que é chamado de âncora de bloco
- Um índice primário é esparsos, uma vez que ele inclui uma entrada para cada bloco de disco do arquivo de dados e as chaves de seu registro de âncora em vez de cada valor de pesquisa (ou cada registro)



Índice primário no campo de chave de ordenação

Figure 14.1

Primary index on the ordering key field of the file shown in Figure 13.7.



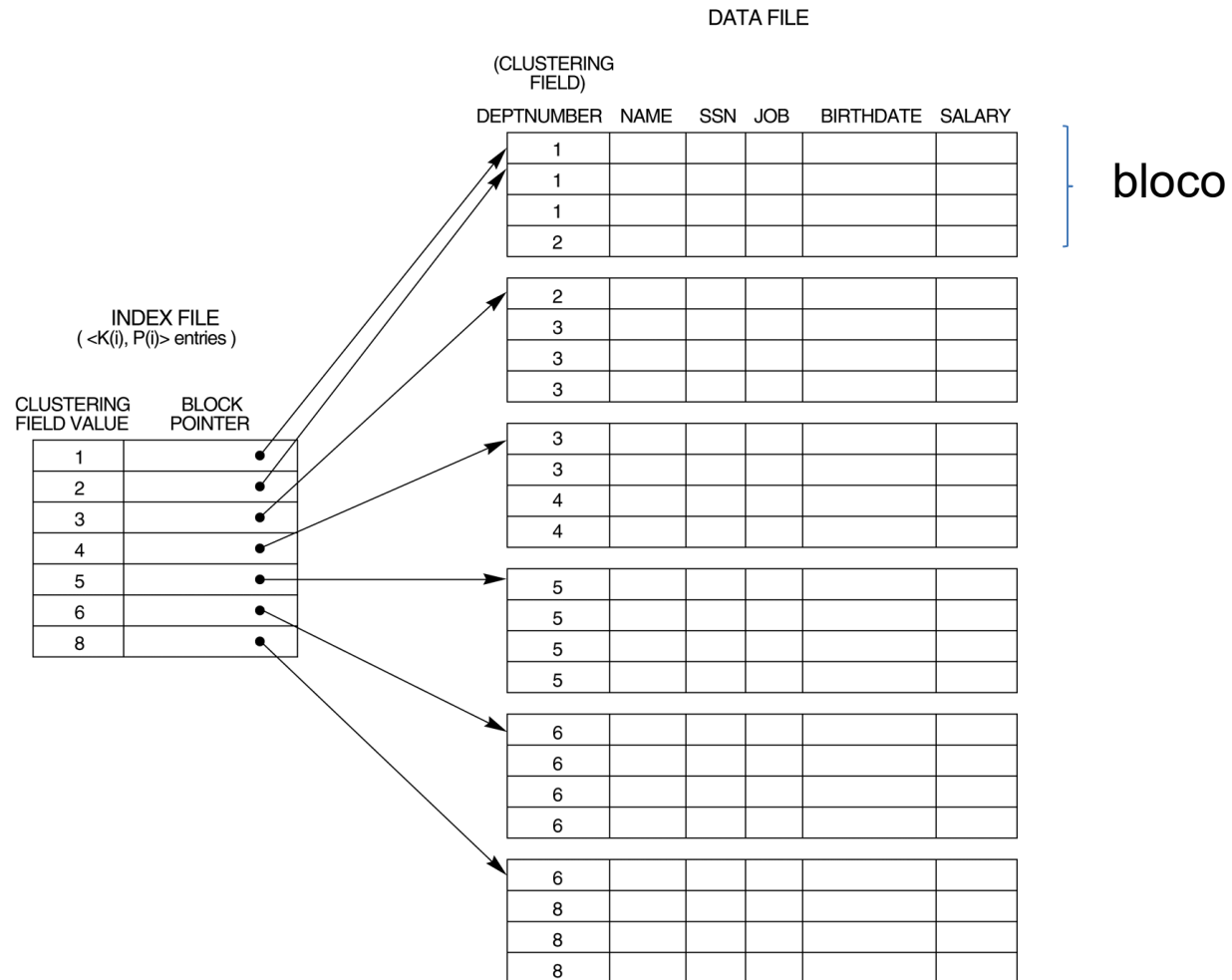
Índices de agrupamento

- Definido sobre um arquivo de dados ordenado
- O arquivo de dados está ordenado fisicamente em um campo **não chave** (que não tem um valor distinto para cada registro)
- Inclui uma entrada de índice para cada valor distinto do campo
- A entrada de índice aponta para o primeiro bloco no arquivo de dados que contém um registro com esse valor
- Esse é outro exemplo de índice esparsos, pois ele tem uma entrada para cada valor distinto do campo de índice, que é uma não chave por definição e, por tanto, tem valores duplicados



INSTITUTO FEDERAL

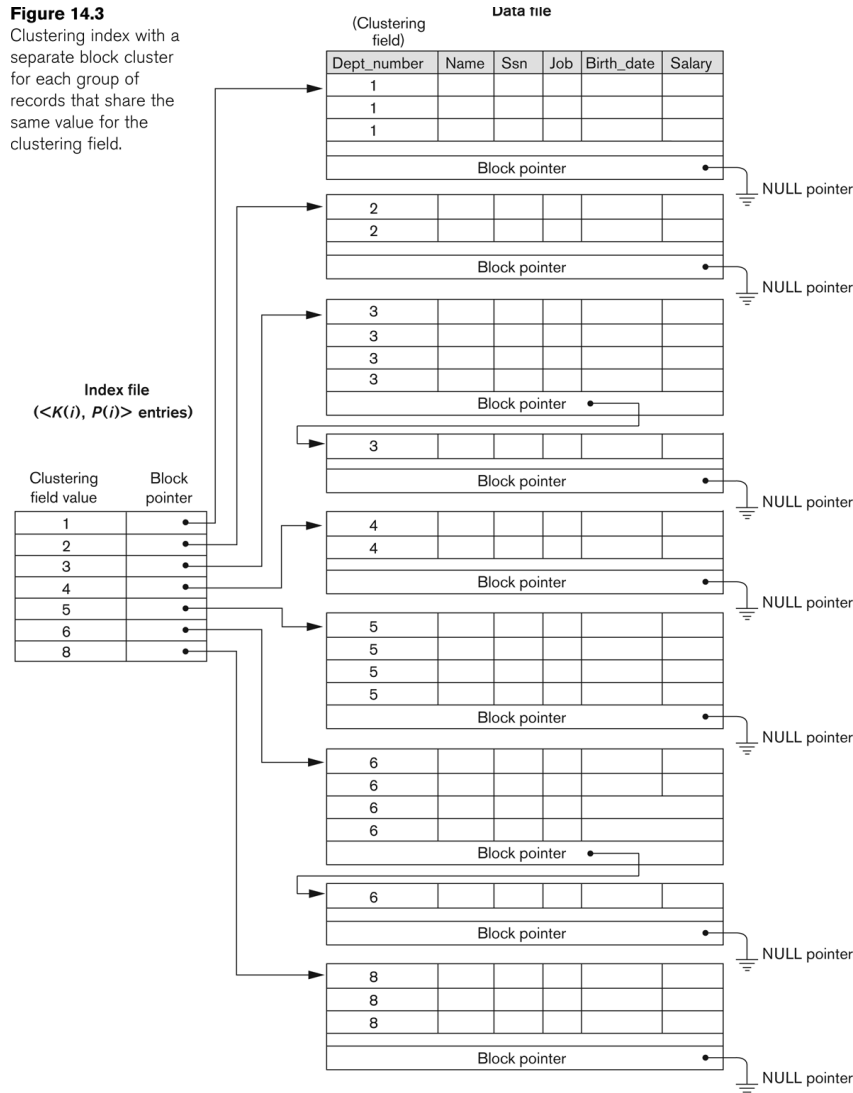
Rio Grande do Sul



Outro exemplo de índice de agrupamento

Figure 14.3

Clustering index with a separate block cluster for each group of records that share the same value for the clustering field.



Índice secundário

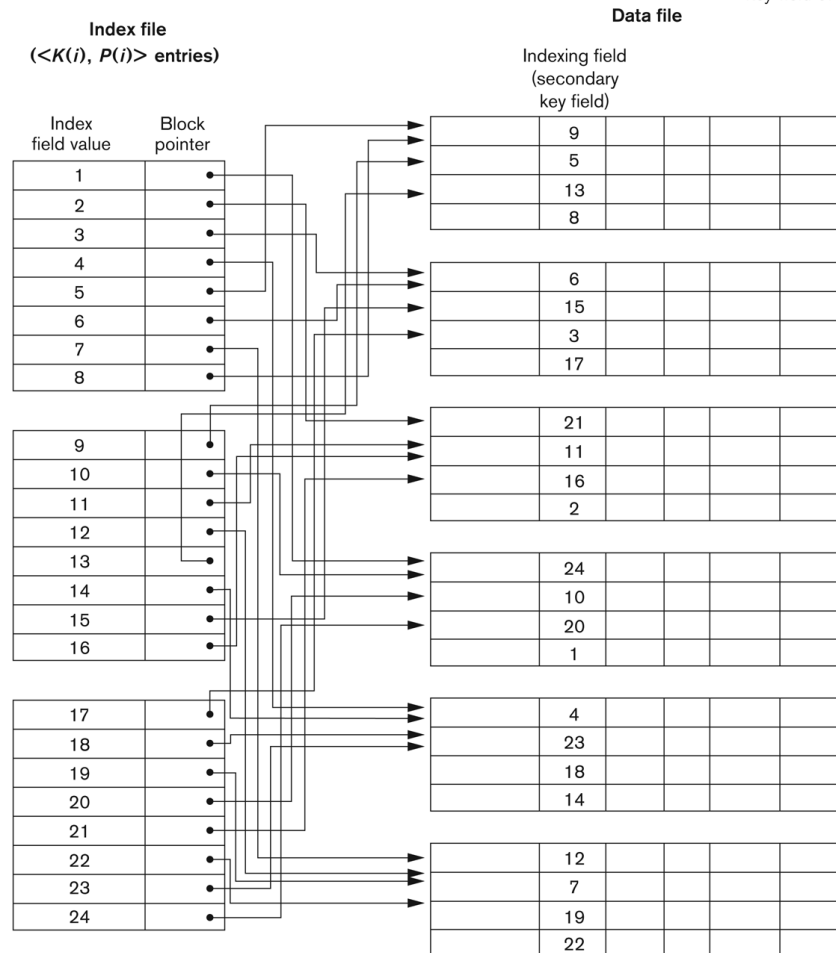
- Um índice secundário fornece uma forma secundária de acesso a um arquivo para o qual já existe um acesso primário
- O índice secundário pode ser em um campo que é uma chave candidata e tem um valor único em cada registro, ou uma não chave com valores duplicados
- O índice é um arquivo ordenado com dois campos
 - O primeiro campo (campo de índice) é do mesmo tipo de dado que algum **campo não ordenado** do arquivo de dados
 - O segundo campo é ou um ponteiro de bloco ou um ponteiro de registro
 - Podem haver muitos índices secundários (e, portanto, campos de índice) para o mesmo arquivo



Exemplo de um índice secundário denso

Figure 14.4

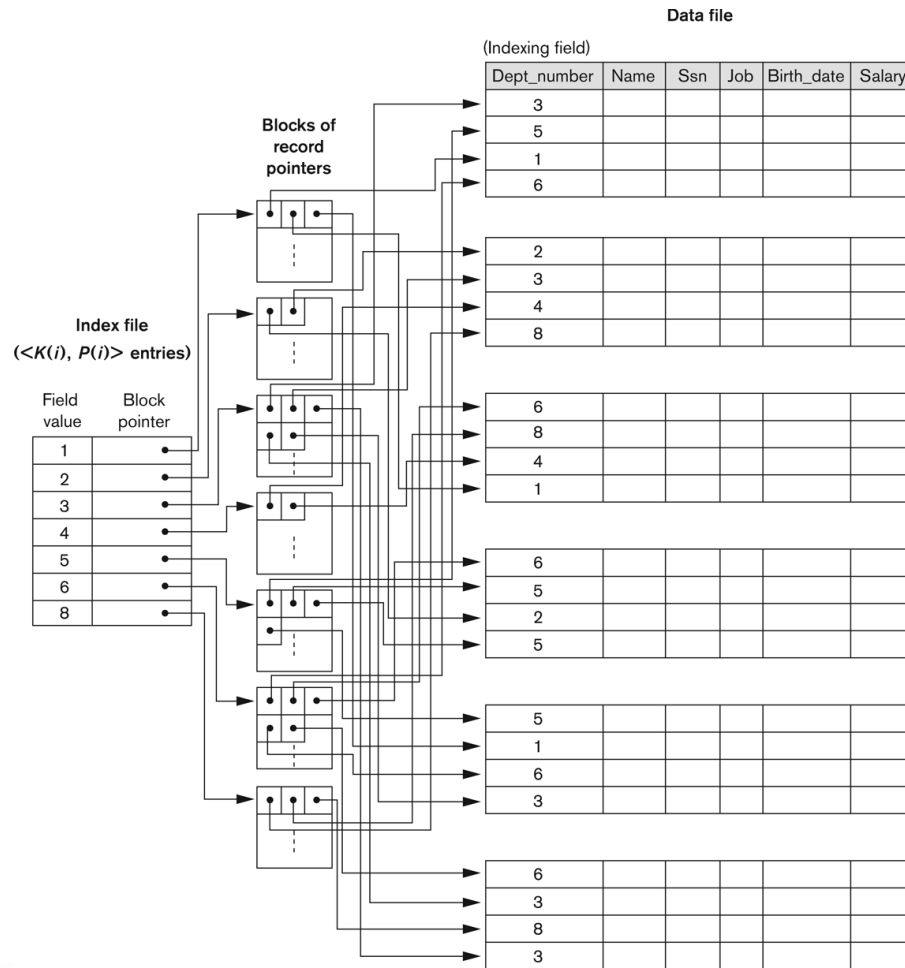
A dense secondary index (with block pointers) on a nonordering key field of a file.



Ponteiro
de bloco

Campo
chave

Exemplo de um Índice Secundário



Ponteiro de registro

Campo não chave

Figure 14.5

A secondary index (with record pointers) on a nonkey field implemented using one level of indirection so that index entries are of fixed length and have unique field values.

Propriedades dos Tipos de Índices

Tipo de Índice	Número de Entradas de índice (Primeiro nível)	Denso ou Esperso	Âncora ¹
Primário	Número de blocos no arquivo de dados	Esperso	Sim
Agrupamento	Número de valores distintos do campo indexado	Não denso	Sim ou não ²
Secundário (campo chave)	Número de registros no arquivo de dados	Denso	Não
Secundário (campo não chave)	Número de registros ou número de valores distintos do campo indexado	Denso ou esperso	Não

¹ Se o índice aponta apenas para o primeiro registro do bloco.

² Sim, se todos os valores distintos do campo indexado iniciam em um novo bloco. Não, caso contrário.

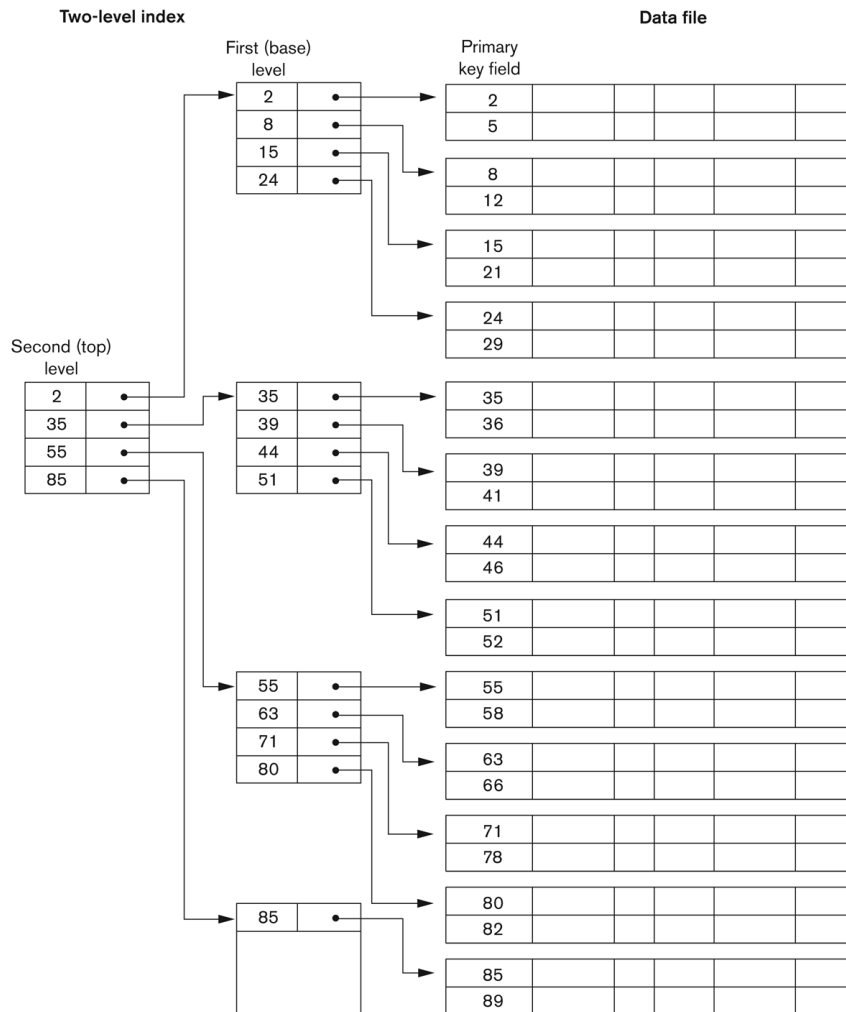


Índices de múltiplos níveis

- Uma vez que o índice de único nível é um arquivo ordenado, nós podemos criar um índice primário para o próprio índice
 - Neste caso, o arquivo de índice original é chamado de índice de **primeiro nível** (ou **base**) e o índice do índice é chamado de índice de **segundo nível**
- Nós podemos repetir o processo, criando um terceiro, quarto, ..., nível de topo, até que todas as entradas do nível de índice de topo caibam em um único bloco de disco
- Um índice multinível pode ser criado para qualquer tipo de índice de primeiro nível (primário, secundário, agrupamento) desde que o índice de primeiro nível contenha mais de um bloco de disco



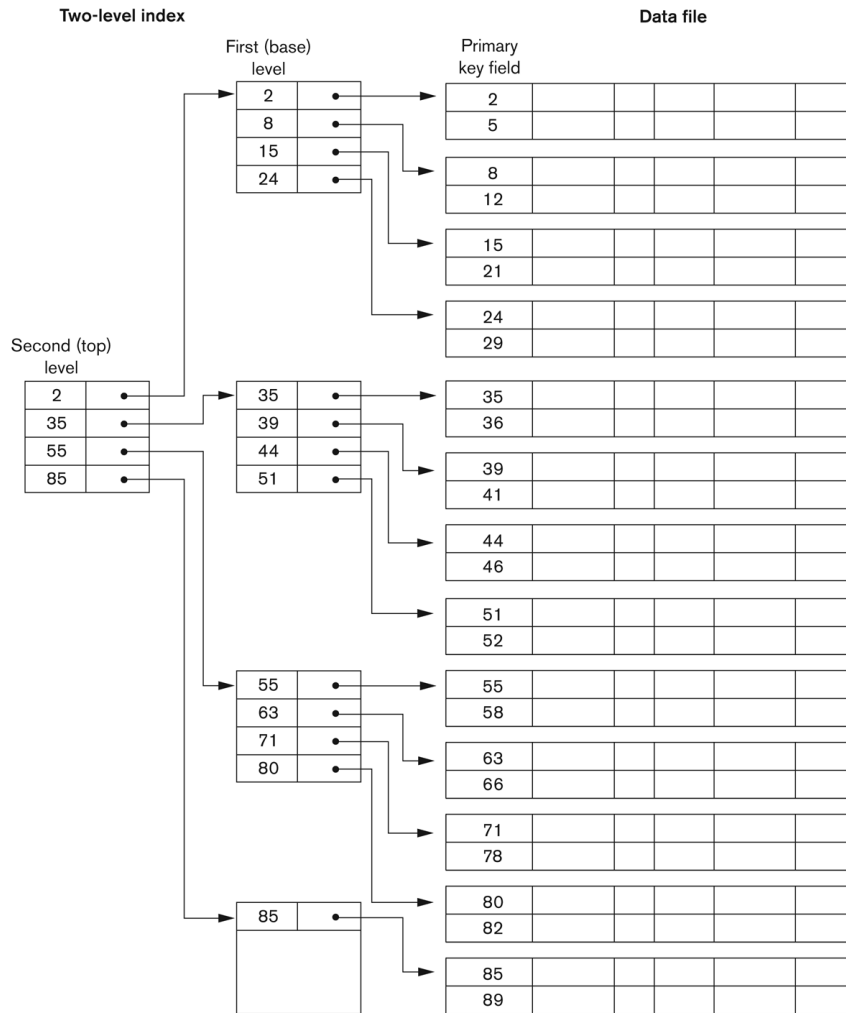
Um índice primário de dois níveis



Por que cada bloco do arquivo de dados contém menos registros que os blocos dos arquivos de índice?

Figure 14.6
A two-level primary index resembling ISAM (Index Sequential Access Method) organization.

Um índice primário de dois níveis

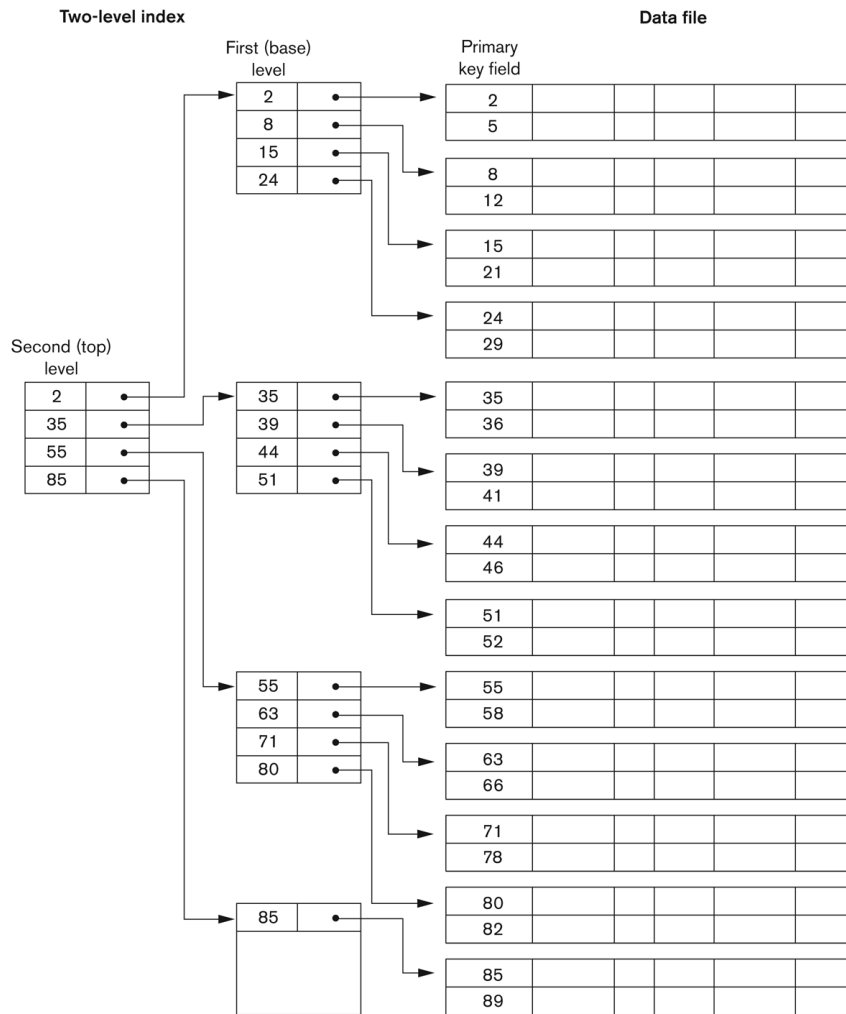


Por que cada bloco do arquivo de dados contém menos registros que os blocos dos arquivos de índice?

Porque o tamanho dos registros dos arquivos de índice é menor que o tamanho dos registros dos arquivos de dados.

Figure 14.6
A two-level primary index resembling ISAM (Index Sequential Access Method) organization.

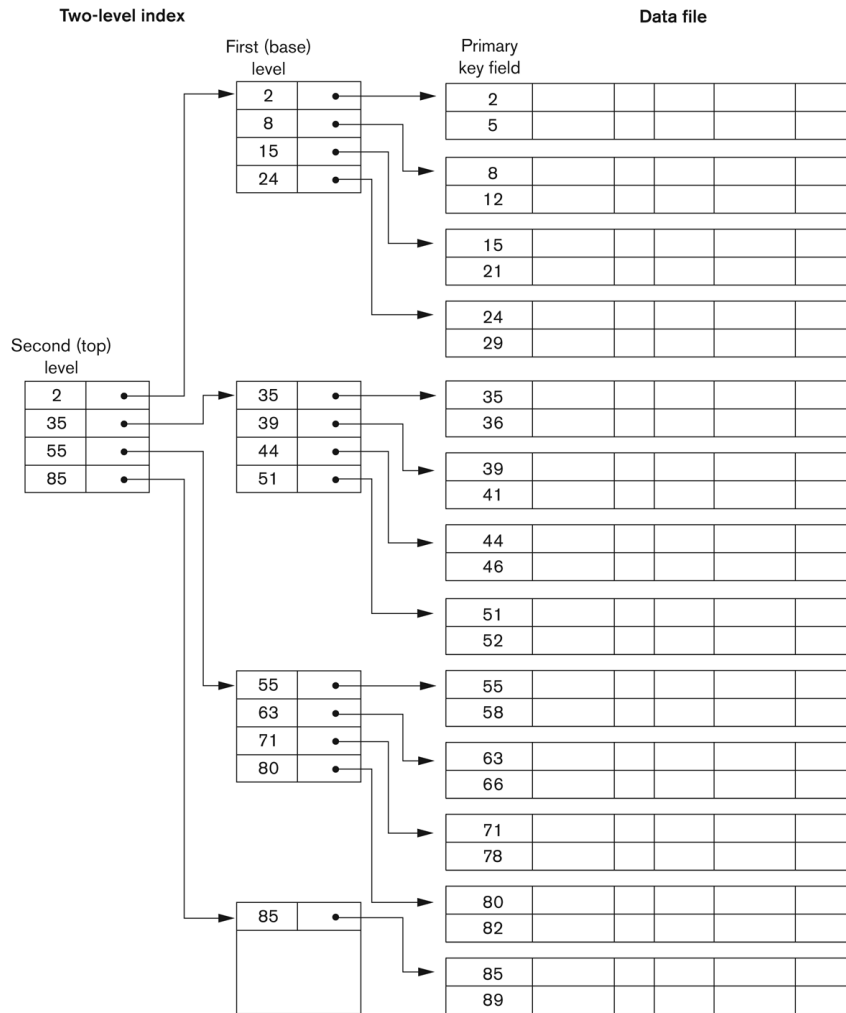
Um índice primário de dois níveis



Quanto blocos de disco são necessários para encontrar o valor 78 usando a **pesquisa linear**?

Figure 14.6
A two-level primary index resembling ISAM (Index Sequential Access Method) organization.

Um índice primário de dois níveis

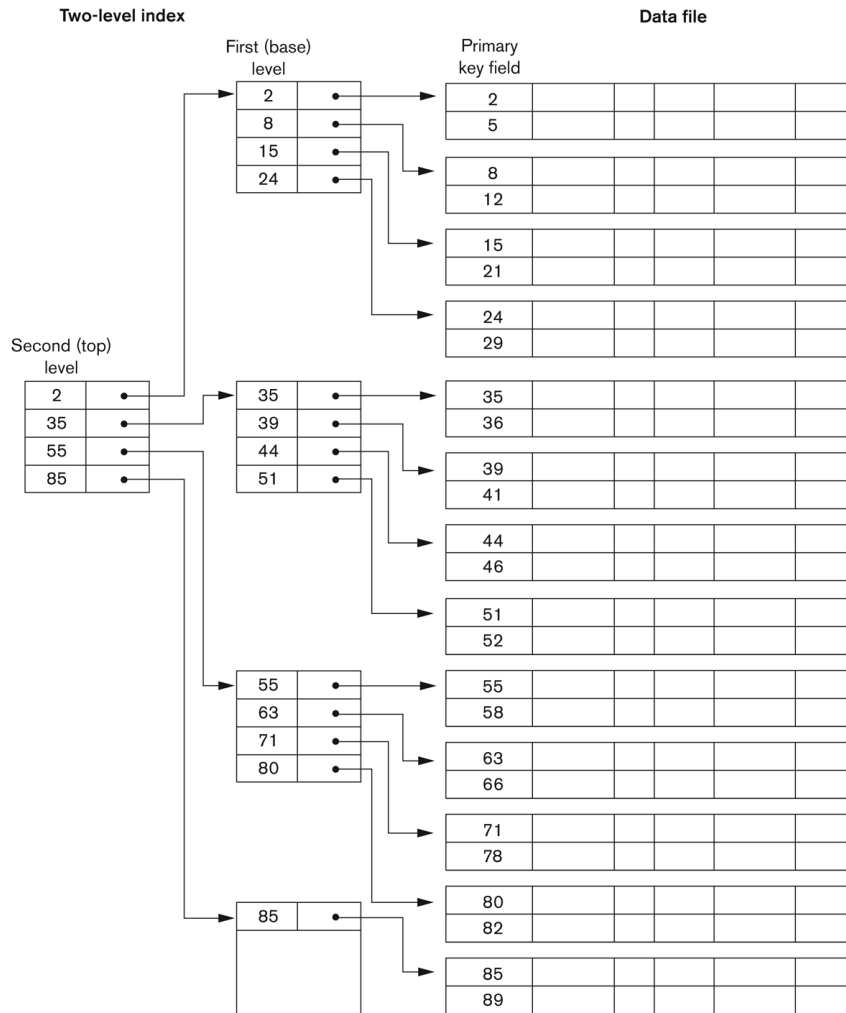


Quanto blocos de disco são necessários para encontrar o valor 78 usando a **pesquisa linear**?

11

Figure 14.6
A two-level primary index resembling ISAM (Index Sequential Access Method) organization.

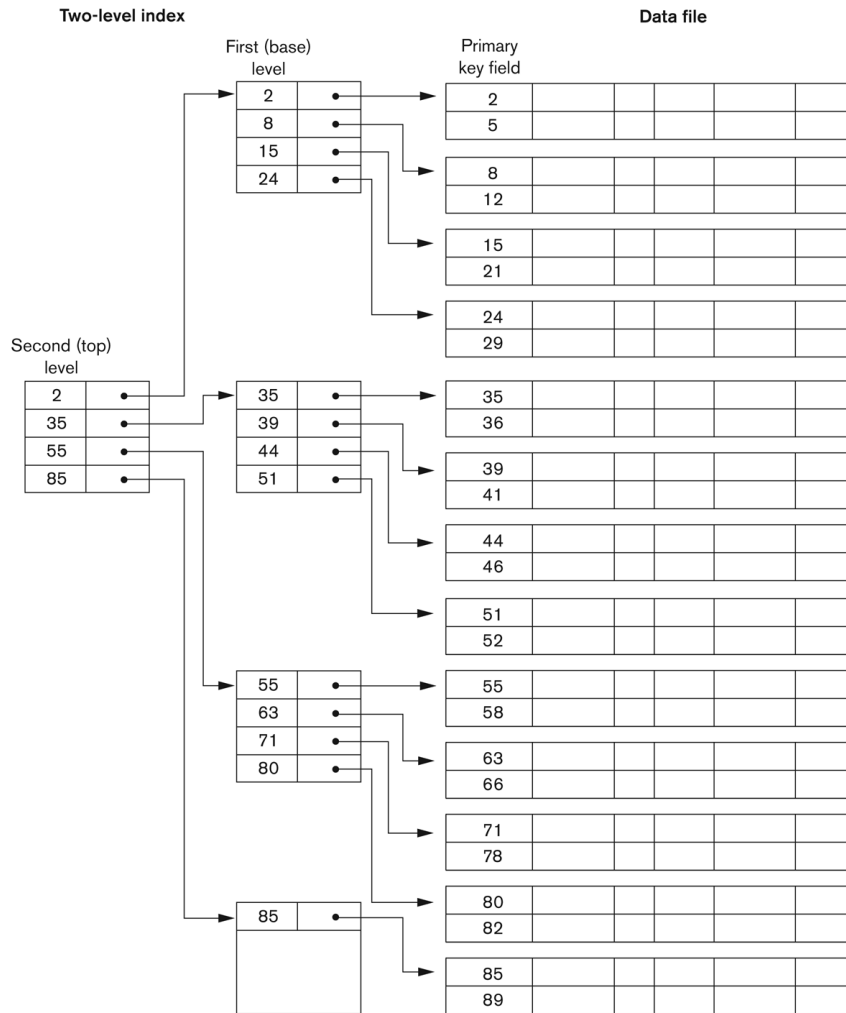
Um índice primário de dois níveis



Quanto blocos de disco são necessários para encontrar o valor 78 usando esse índice?

Figure 14.6
A two-level primary index resembling ISAM (Index Sequential Access Method) organization.

Um índice primário de dois níveis



Quanto blocos de disco são necessários para encontrar o valor 78 usando esse índice?

3

Figure 14.6
A two-level primary index resembling ISAM (Index Sequential Access Method) organization.

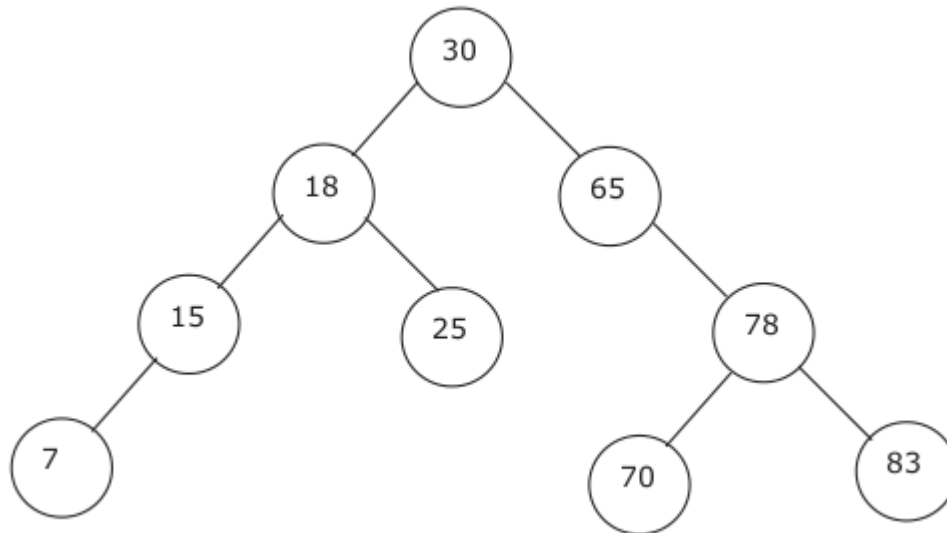
Índices multiníveis

- Um índice multinível é uma forma de *árvore de pesquisa*
 - Entretanto, a inserção e a exclusão de novas entradas de índice é um problema grave porque cada nível de um índice é um arquivo ordenado



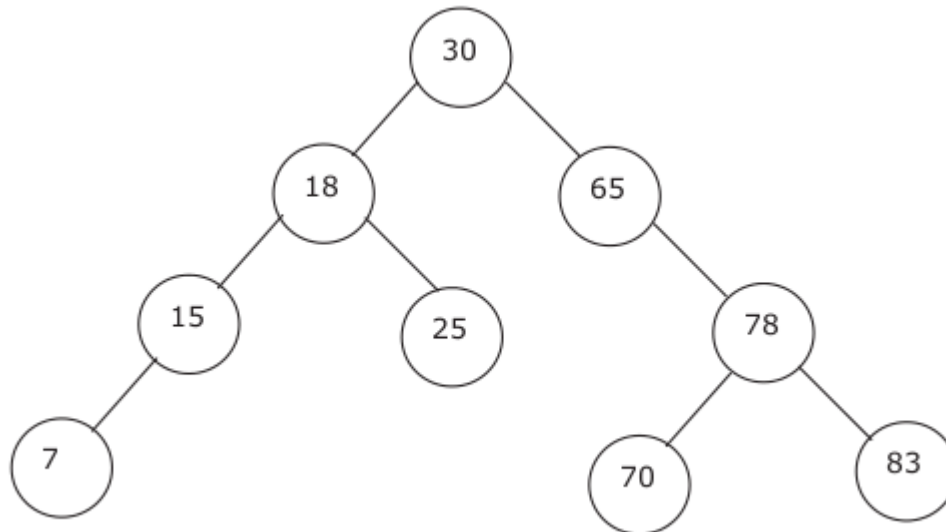
Árvores: Conceitos

- Nó
 - Elemento da árvore



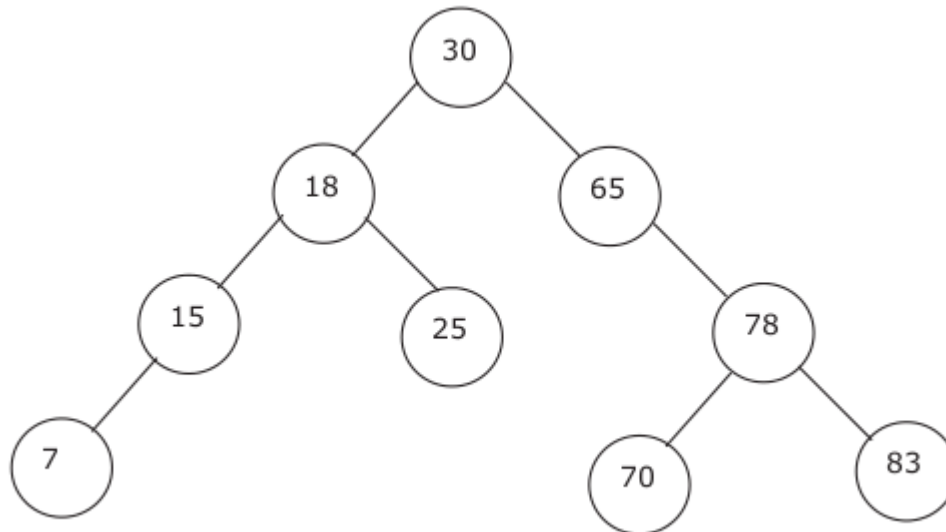
Árvores: Conceitos

- Raiz



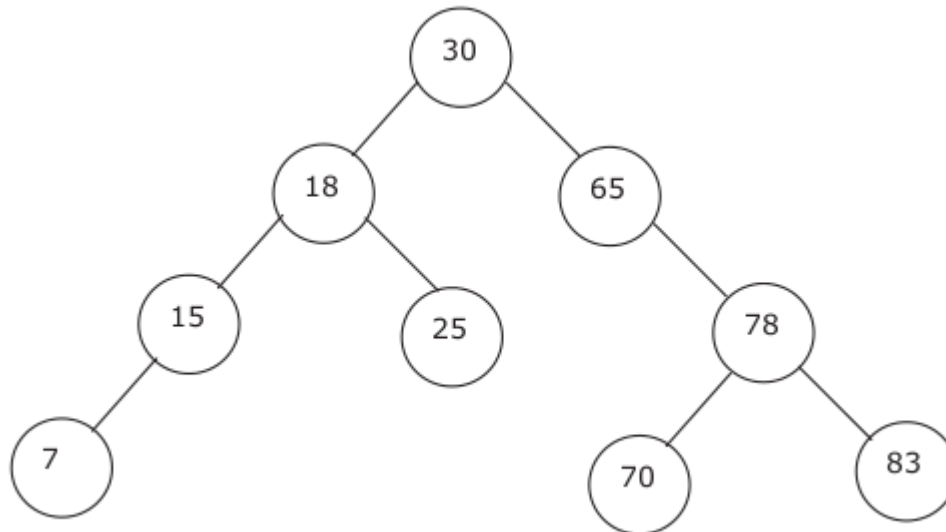
Árvores: Conceitos

- Nó pai



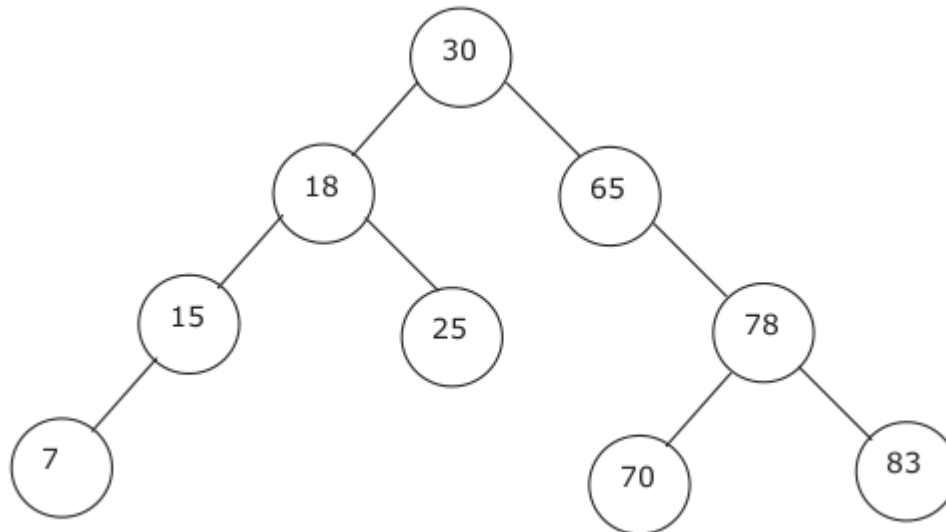
Árvores: Conceitos

- Nó filho



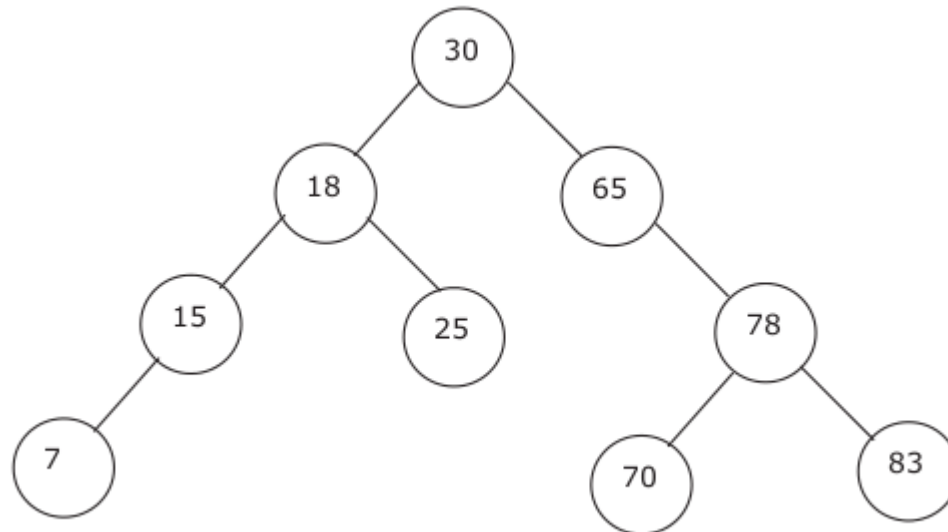
Árvores: Conceitos

- Nó folha
 - Nó terminal
 - Não tem filhos



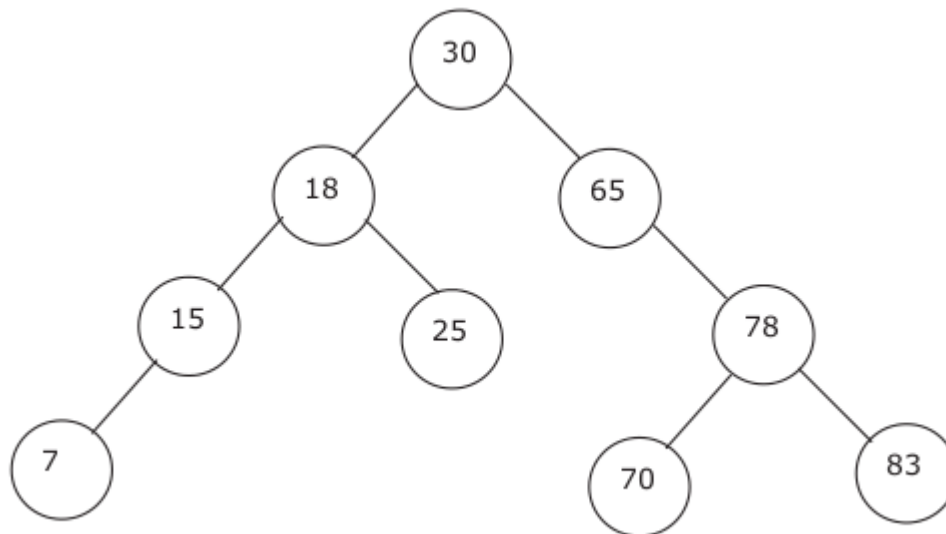
Árvores: Conceitos

- Nó interno
 - Não é terminal
 - Tem filhos



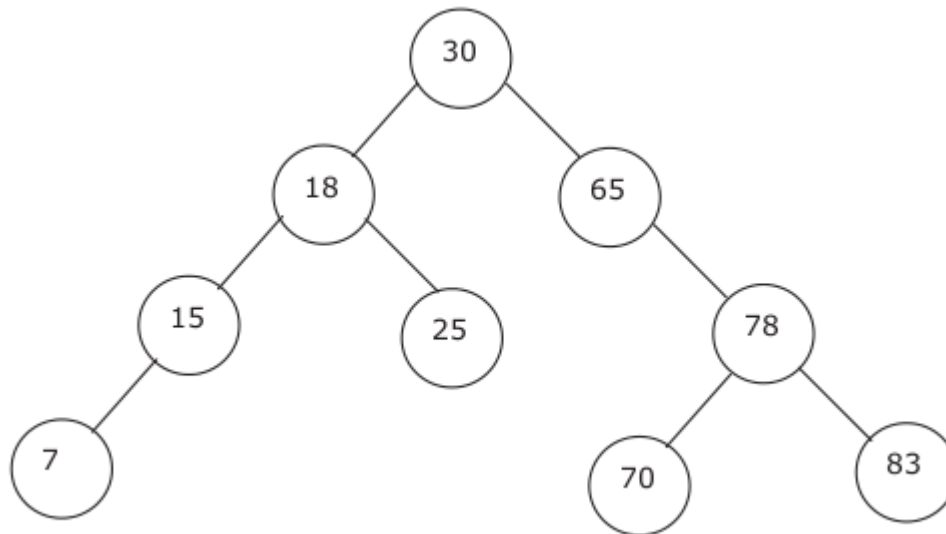
Árvores: Conceitos

- Subávore



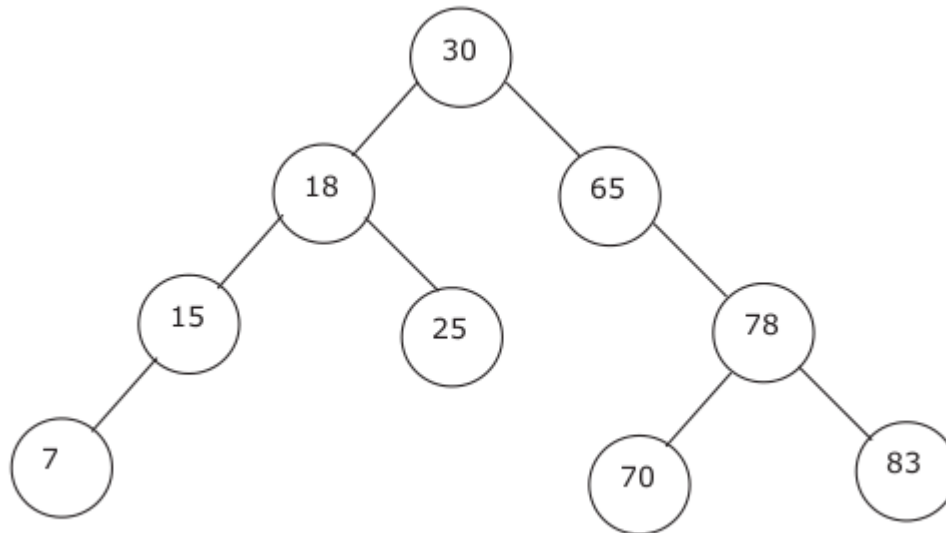
Árvores: Conceitos

- Nível
 - O nível do nó raiz é **0** (ou **1** → conforme autor)
 - O nível de um nó = o nível de seu pai + 1



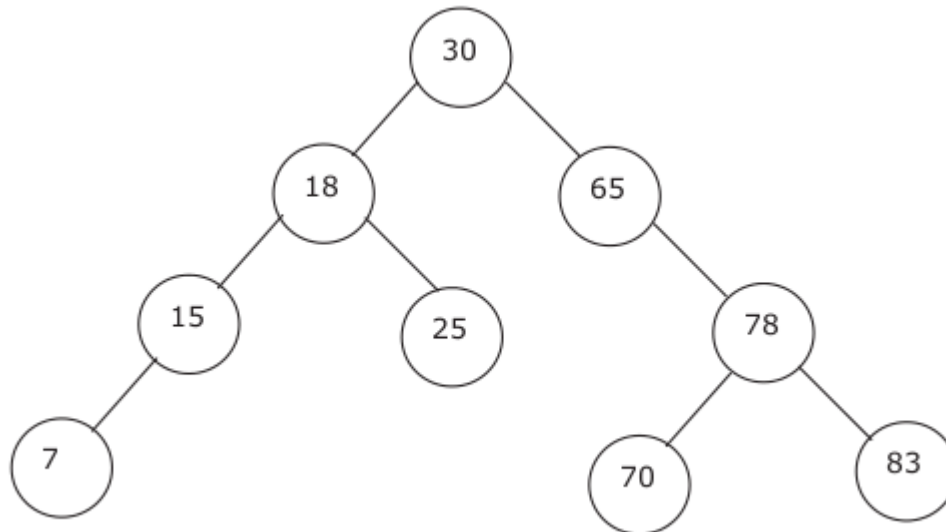
Árvores: Conceitos

- Profundidade ou altura da árvore
 - É o nível máximo de seus nós + 1 (se $\text{nível}(\text{raiz}) = 0$)



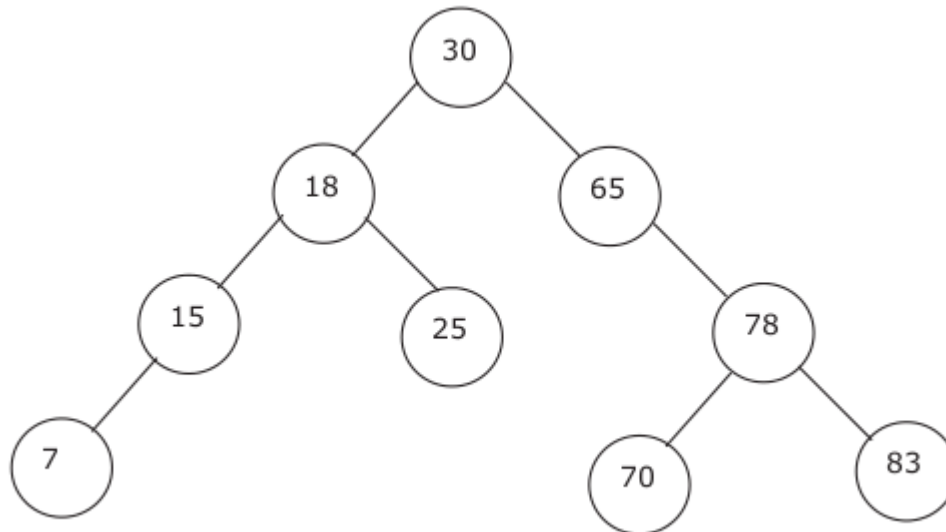
Árvores: Conceitos

- Grau de um nó
 - Número de filhos

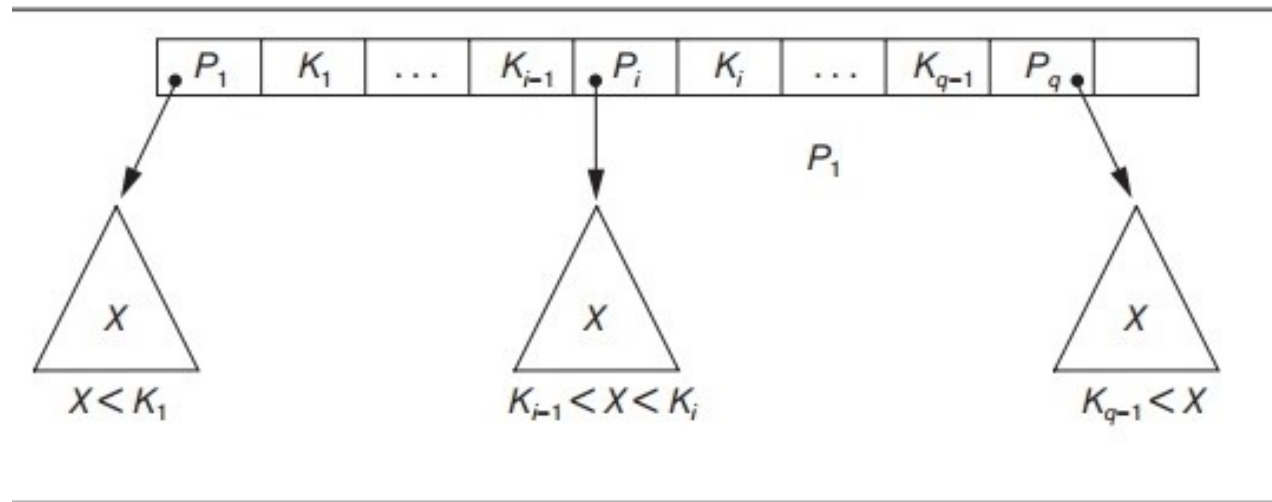


Árvores: Conceitos

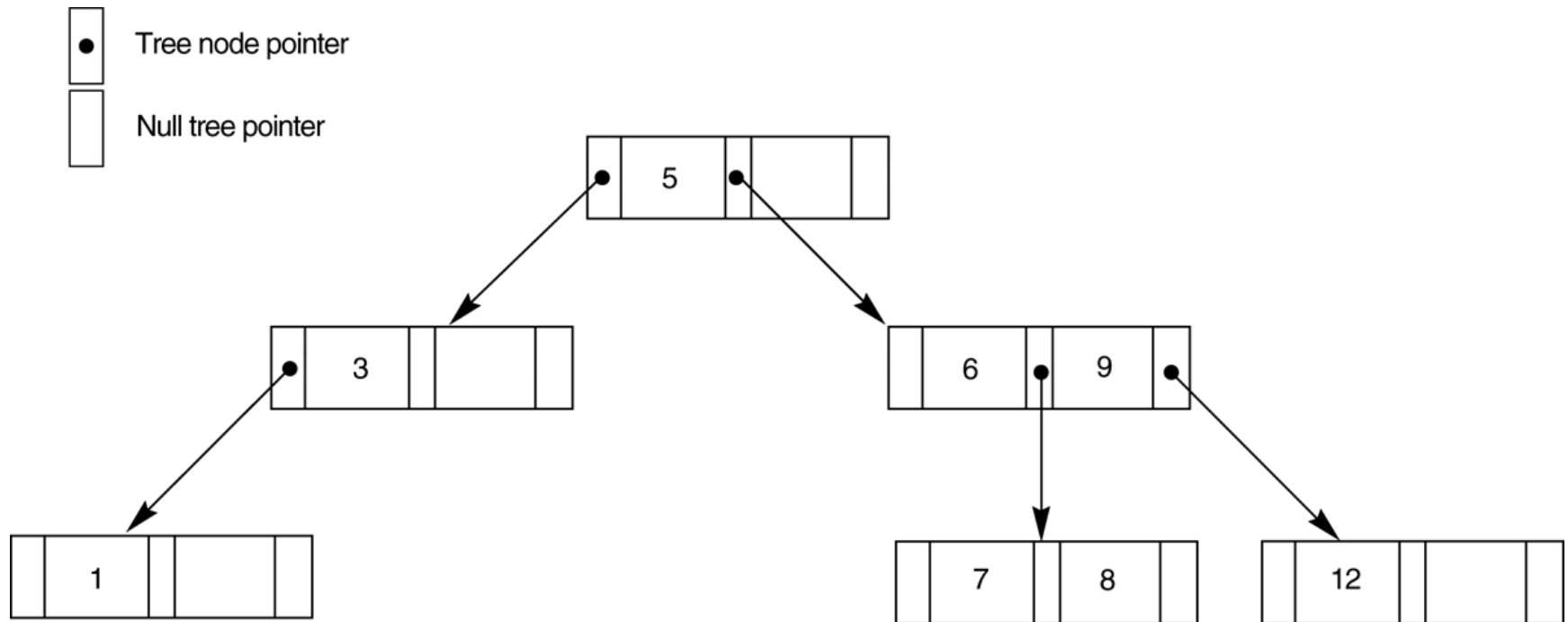
- Ordem ou grau da árvore
 - Número máximo de filhos em um nodo



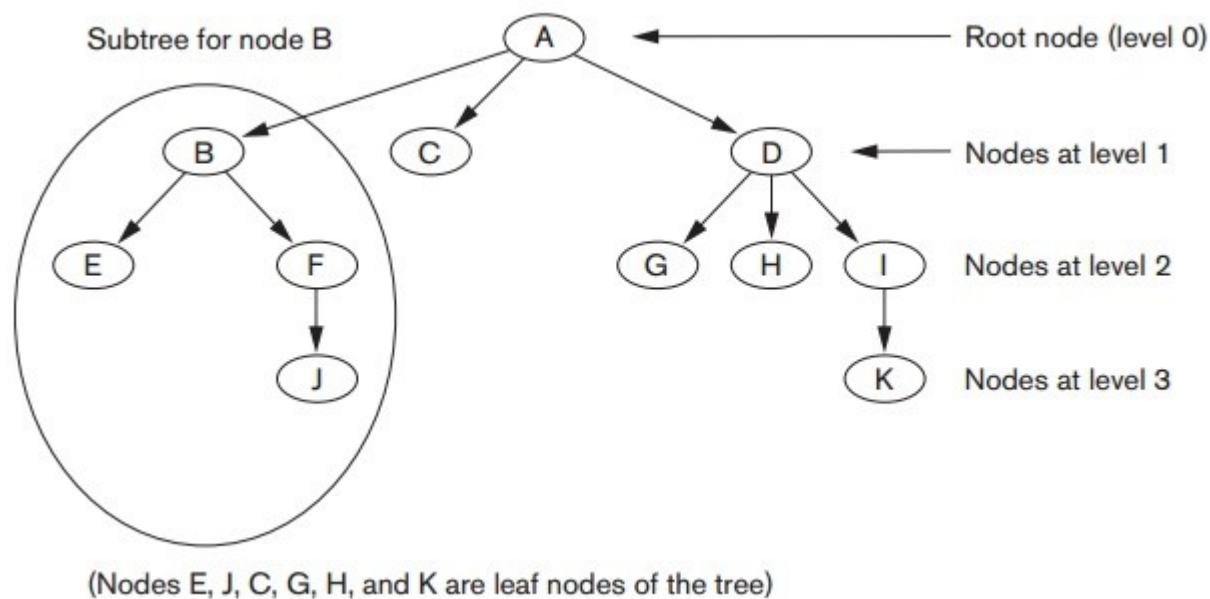
Um nodo em uma árvore de pesquisa com ponteiros para as suas subárvores



Uma árvore de pesquisa de ordem $p = 3$



Exemplo de árvore desbalanceada



Objetivos para balancear uma árvore de pesquisa

- Garantir que os nós sejam distribuídos por igual, de modo que a profundidade da árvore seja minimizada para determinado conjunto de chaves e que a árvore não fique distorcida com alguns nós em níveis muito profundos
- Tornar a velocidade de pesquisa uniforme, de modo que o tempo médio para encontrar qualquer chave aleatória seja aproximadamente o mesmo

Índices multiníveis dinâmicos usando B Trees e B+ Trees

- A maioria dos índices multiníveis usam as estruturas de dados B tree ou B+ tree devido ao problema da inserção e exclusão
 - Isso deixa espaço em cada nodo da árvore (bloco de disco) para permitir novas entradas de índice
- Essas estruturas de dados são variações das árvores de pesquisa que permitem inserção e exclusão de novos valores de pesquisa eficientes
- Nas estruturas de dados B Tree e B+ Tree, cada nodo corresponde a um bloco de disco



Índices multiníveis dinâmicos usando B Trees e B+ Trees

- Embora minimizar o número de níveis na árvore seja um objetivo, outro objetivo implícito é garantir que a árvore de índice não precise de muita reestruturação quando os registros são inseridos e excluídos do arquivo de dados
- Assim, queremos que os nós sejam os mais cheios possíveis e não queremos que quaisquer nós sejam vazios se houver muitas exclusões
- A exclusão de registro pode deixar alguns nós na árvore quase vazios, desperdiçando assim o espaço de armazenamento e aumentando o número de níveis
- A árvore B resolve esses dois problemas, especificando restrições adicionais na árvore de pesquisa



Árvore B

- A árvore B tem restrições adicionais que garantem que a árvore sempre esteja balanceada e que o espaço desperdiçado pela exclusão, se houver, nunca se torne excessivo
- Os algoritmos para inserção e exclusão, porém, tornam-se mais complexos a fim de manter essas restrições
- Apesar disso, a maioria das inserções e exclusões são processos simples. Elas tornam-se complicadas somente sob circunstâncias especiais
 - Se um nodo está cheio a inserção causa uma divisão em dois nodos
 - Essa divisão pode se propagar para outros níveis da árvore
- Uma exclusão é muito eficiente se não tornar o nodo menos que meio cheio
 - Se uma exclusão tornar o nodo menos que **meio cheio**, ele deve ser mesclado com seus nodos vizinhos



Árvore B de ordem 4

P1	K1	P2	K2	P3	K3	P4	K4
----	----	----	----	----	----	----	----

Nós cheio

		P2	K2	P3	K3		
--	--	----	----	----	----	--	--

Nós meio cheio

Diferença entre árvores B e B+

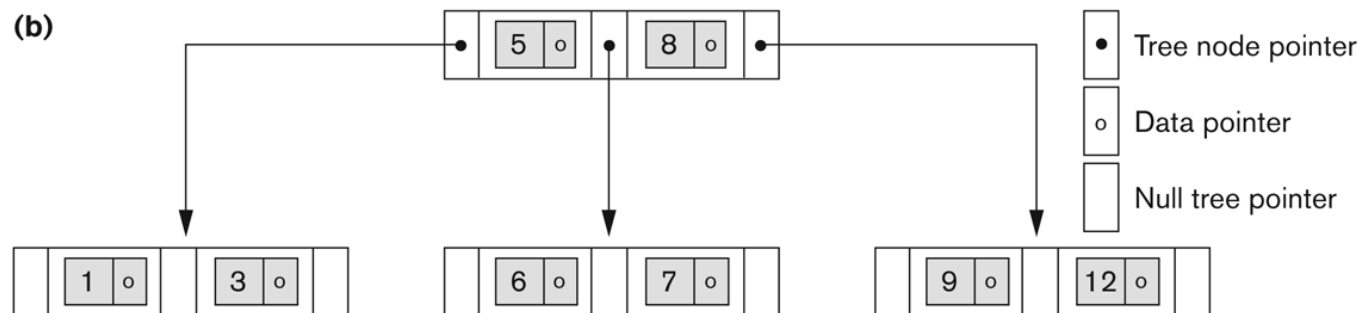
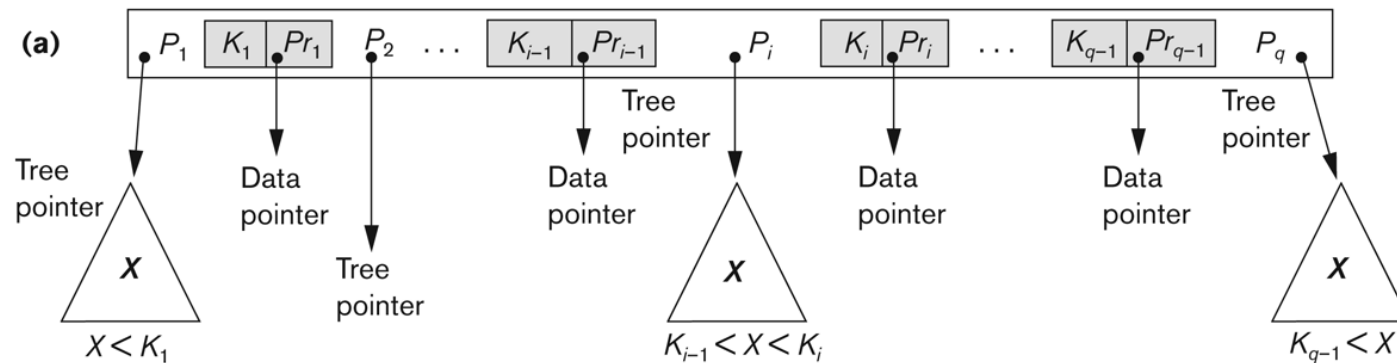
- A maioria das implementações de um índice multinível dinâmico utiliza uma variação da árvore B chamada árvore B+
- Em uma árvore B, cada valor do campo de pesquisa aparece uma vez em algum nível na árvore, junto com um ponteiro de dados
- Em uma árvore B+, os ponteiros de dados são armazenados apenas nos nós *folha* da árvore, logo, a estrutura dos nós folha difere da estrutura dos nós internos
- Uma árvore B+ pode ter menos níveis (ou maior capacidade de valores de pesquisa) que a árvore B correspondente



Exemplos de árvore B

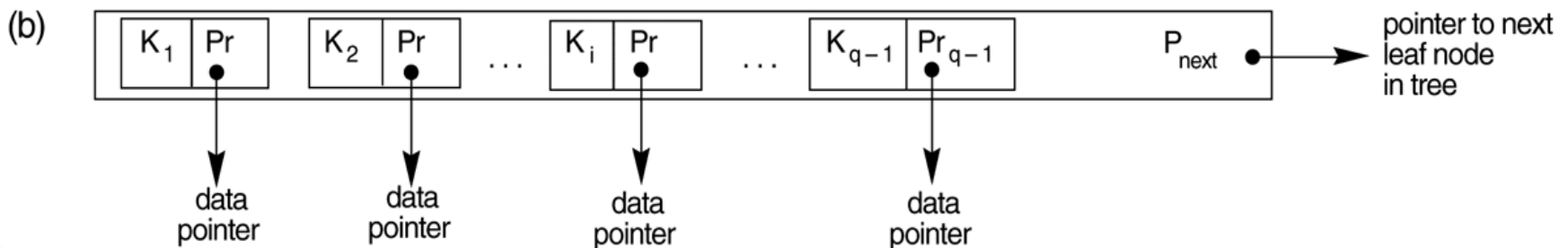
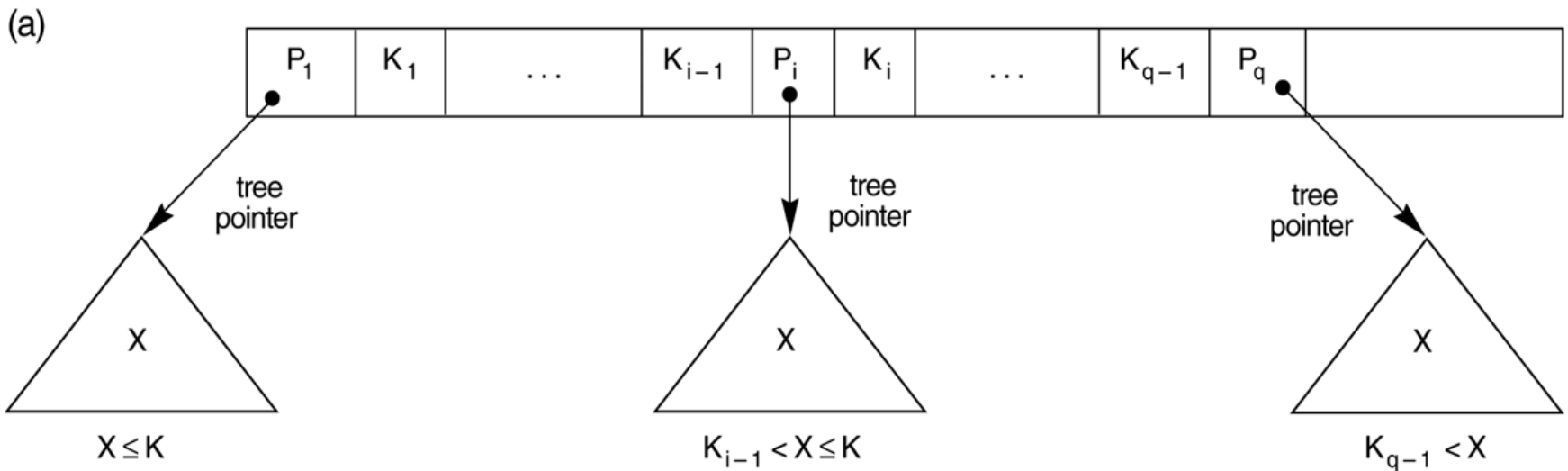
Figure 14.10

B-Tree structures. (a) A node in a B-tree with $q - 1$ search values. (b) A B-tree of order $p = 3$. The values were inserted in the order 8, 5, 1, 7, 3, 12, 9, 6.



Os nodos de uma árvore B+

- a) Nodo interno de uma árvore B+ com $q - 1$ valores de pesquisa
- b) Nodo folha de uma árvore B+ com $q - 1$ valores de pesquisa e $q - 1$ ponteiros de dados.



Exemplos

- Árvore B
 - <https://www.cs.usfca.edu/~galles/visualization/BTree.html>
- Árvore B+
 - <https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>



Exemplo de busca em uma B+ tree

- <https://i.makeagif.com/media/10-06-2015/DDVEX.gif>

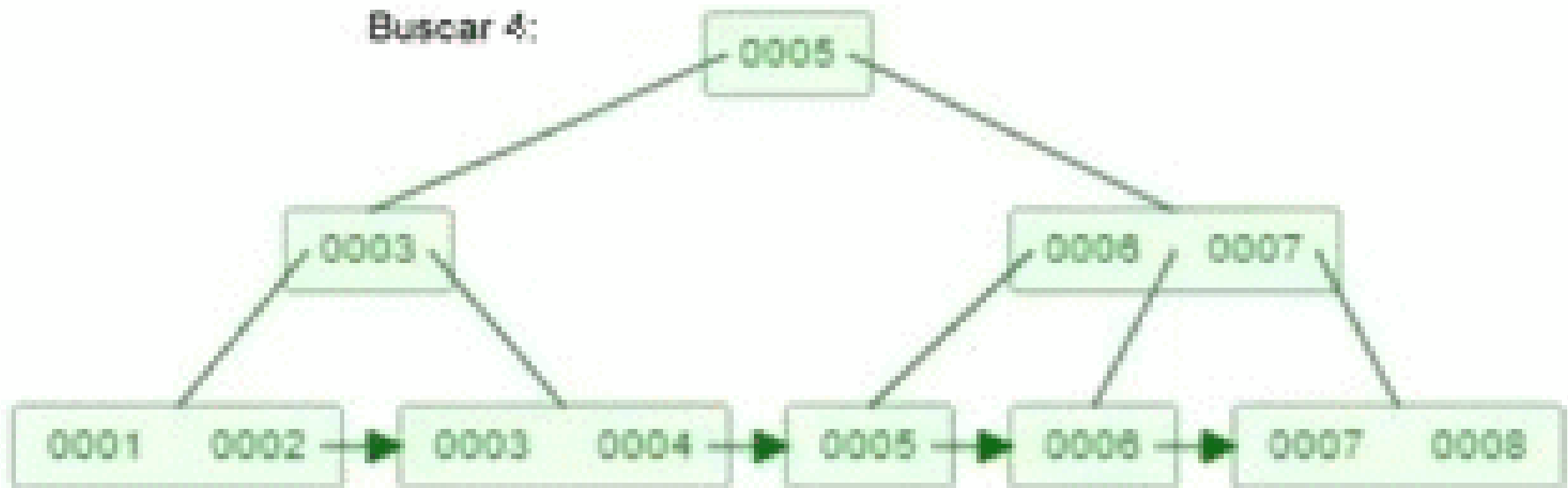
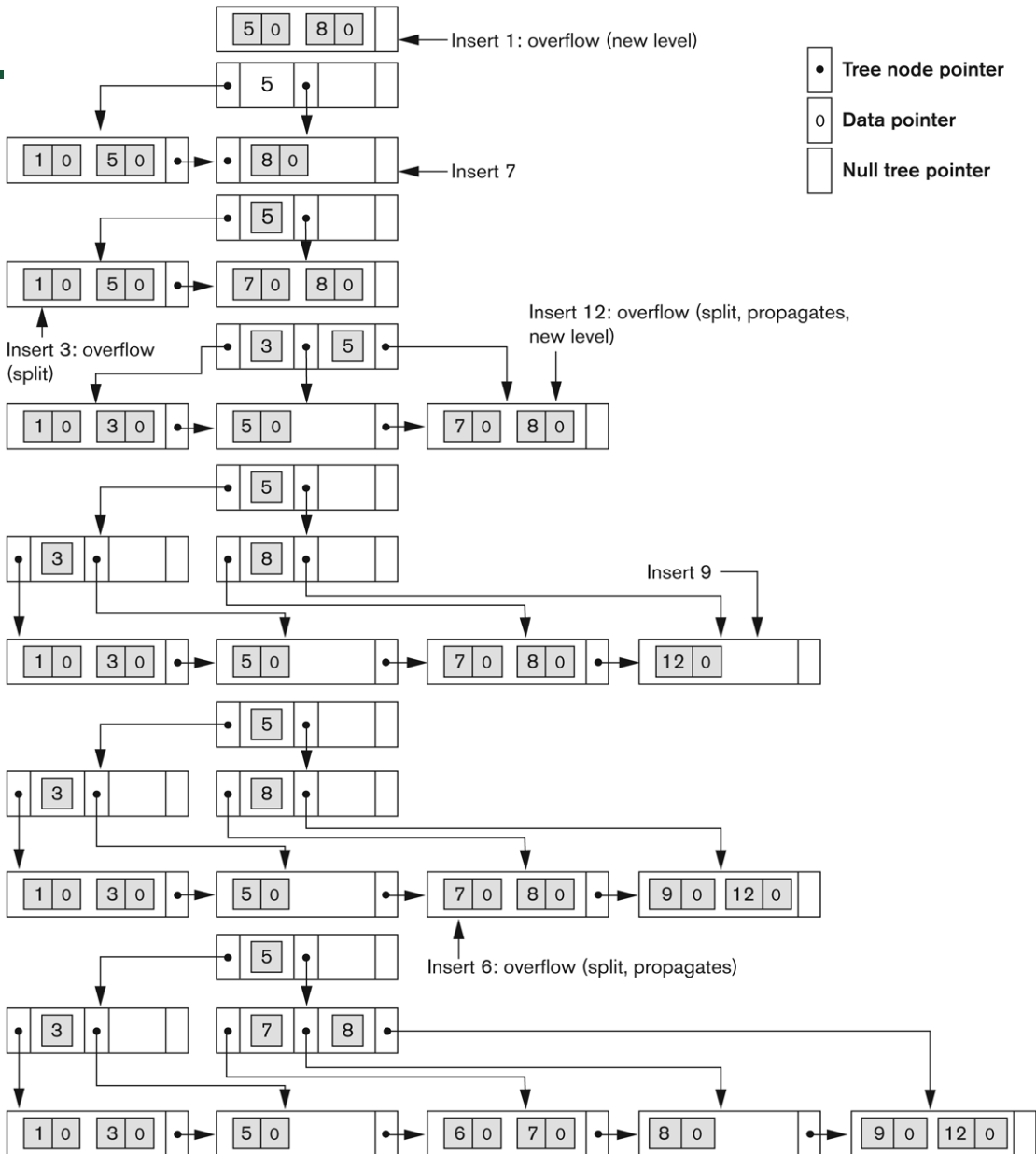


Figure 14.12

An example of insertion in a B⁺-tree with $p = 3$ and $p_{\text{leaf}} = 2$.

Insertion sequence: 8, 5, 1, 7, 3, 12, 9, 6



Deletion sequence: 5, 12, 9

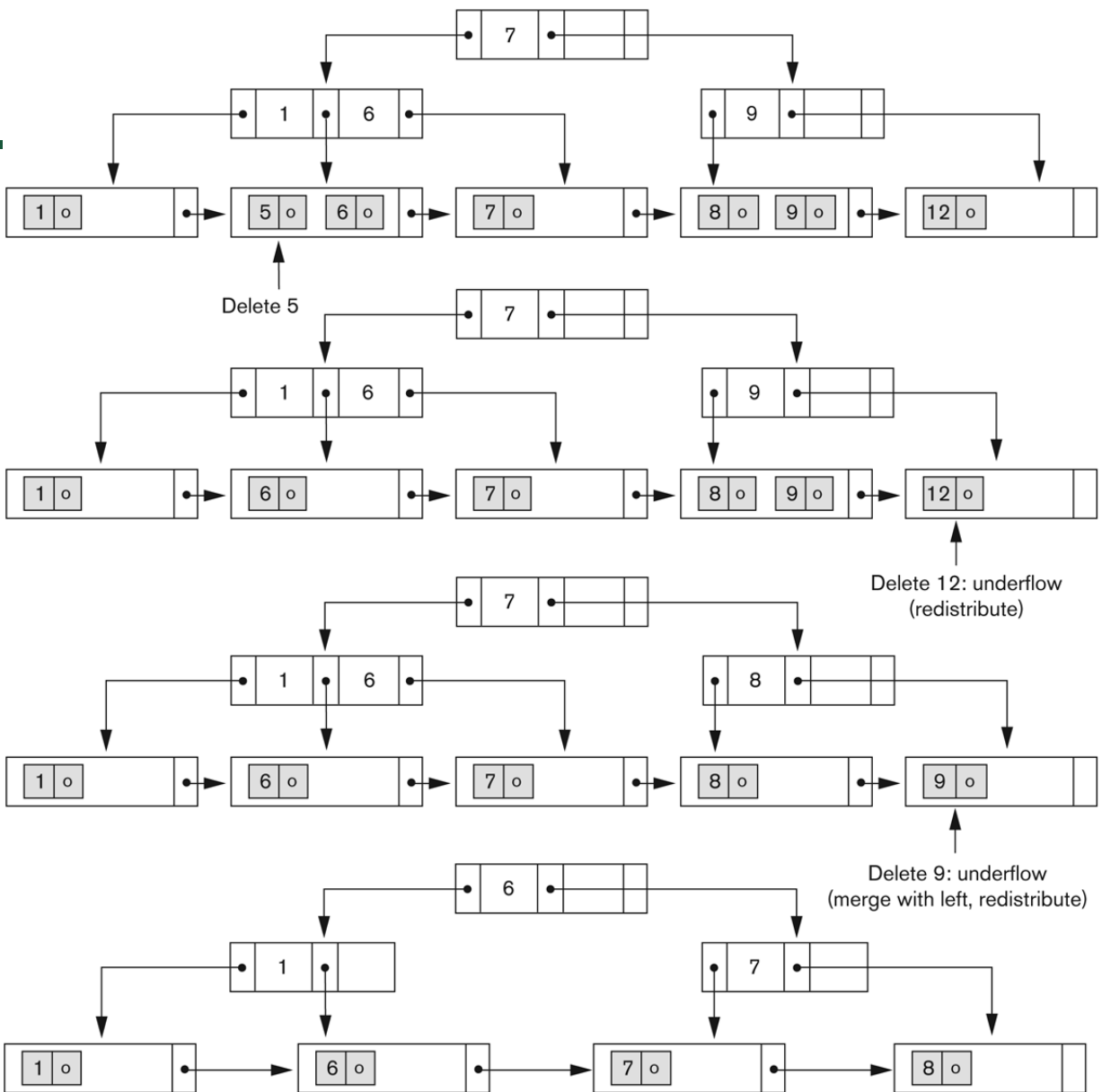


Figure 14.13

An example of deletion from a B⁺-tree.

Outro exemplo de exclusão

- <https://i.makeagif.com/media/10-06-2015/6FxETL.gif>

Eliminar 50:

